Université Abdelmalek Essaâdi

École Nationale des Sciences Appliquées

Al-Hoceima

**Filière : Transformation Digitale et Intelligence Artificielle 2**

# Rapport TP 3

## Applications web avancées avec Java et Spring

Mise en place de l'authentification et gestion de sessions web

**Réalisé par :**

Moaad Daoudi

**Encadré par :**

Pr. Mohamed Cherradi

Année Universitaire 2025/2026

# Table des matières

**Introduction** **3**

**1 Stratégie et Persistance des Données** **4**
1.1 Architecture Globale en multi-couches . . . . . 4
1.2 Modèle de Données (Models) . . . . . 5
1.2.1 Classe `User.java` . . . . . 5
1.3 Couche d'Accès aux Données (DAO) . . . . . 6
1.3.1 Interface `IUser` et Implémentation `UserDaoImpl` . . . . 6

**2 Sécurité, Services et Sessions** **7**
2.1 Couche Métier (Services) . . . . . 7
2.1.1 `UserService.java` . . . . . 7
2.2 Composants Transversaux : Filtres et Listeners . . . . . 8
2.2.1 Filtre de Sécurité : `AuthFilter.java` . . . . . 8
2.2.2 Écouteur d'Application : `AppListener.java` . . . . . 9
2.3 Couche Contrôleur et Vues Web . . . . . 9
2.3.1 Servlets d'Authentification . . . . . 9
2.3.2 Interfaces Visuelles (JSP) . . . . . 10

**Conclusion** **13**

# Table des figures

1.1 Code source de la classe `User.java` . . . . . 5
1.2 Exemple d'accès aux données via JDBC dans `UserDaoImpl` . . 6

2.1 Classe `UserService` : Logique métier . . . . . 8
2.2 Interception des requêtes avec `AuthFilter` . . . . . 9
2.3 Génération d'une session HTTP par la `LoginServlet` . . . . . 10
2.4 Interfaces d'inscription et de vérification d'identité . . . . . 11
2.5 Le tableau de bord sécurisé (Dashboard) . . . . . 12

# Introduction

## Contexte

Contrairement au TP précédent axé sur l'approche fonctionnelle basique, ce TP3 cible la mise en place concrète de concepts avancés transversaux indispensables au développement logiciel moderne avec Java Enterprise Edition (JEE). L'authentification des utilisateurs, la gestion des sessions HTTP et la sécurité globale de l'application deviennent des enjeux critiques.

## Objectifs du document

L'objectif de ce document est de démontrer l'implémentation robuste d'un système d'authentification. Il y sera question de la persistence des données par une base relationnelle connectée via JDBC, du design pattern DAO, ainsi que des restrictions d'accès gérées par l'intermédiaire de filtres et d'écouteurs d'événements.

# Chapitre 1

# Stratégie et Persistance des Données

## 1.1 Architecture Globale en multi-couches

Pour garantir la maintenabilité et la scalabilité, nous avons adopté une architecture stricte en couches (N-Tiers) qui isole clairement chaque responsabilité de l'application :

- **La Couche Présentation (Vues)** : Conçue via des pages JSP (login, register, dashboard), elle est la seule responsable de l'affichage de l'interface utilisateur et de la capture des interactions.
- **La Couche Contrôleur (Servlets & Filtres)** : Les Servlets interceptent les requêtes HTTP, valident les paramètres de base, et commandent la logique métier. En parallèle, les Filtres agissent comme des barrières de sécurité pour s'assurer que seules les sessions autorisées accèdent aux vues privées.
- **La Couche Métier (Services)** : Représentée par des classes comme `UserService`, elle gère la validation, l'orchestration des données, et applique les règles métiers de l'authentification (ex. vérification d'email) avant d'autoriser tout enregistrement ou accès.
- **La Couche d'Accès aux Données (DAO)** : Destinée uniquement à communiquer avec la base de données. Elle dissimule toute logique de connexion et de formulation de requêtes (SQL via JDBC) au reste de l'application, rendant le couplage matériel abstrait.
- **Les Entités (Models)** : Il s'agit des objets Java (POJOs), comme notre classe `User`, créés pour transiter aisément d'une couche à l'autre en enveloppant l'information issue ou dirigée vers les tables SQL.

Ce flux de travail asynchrone permet de manipuler conjointement une

base de données avec des flux web MVC en toute sécurité, sans utiliser de code "spaghetti".

## 1.2 Modèle de Données (Models)

### 1.2.1 Classe `User.java`

L'entité centrale de ce TP. Elle modélise les données de l'utilisateur final en base. On y retrouve l'identifiant (`id`), le `nom`, l'`email`, et bien sûr le `mot de passe` encapsulés dans un POJO classique, assorti d'un constructeur par défaut et des méthodes accesseurs (Getters) / mutateurs (Setters).

```java
package models;

import java.io.Serializable;

public class User implements Serializable {

    private String nom;
    private String prenom;
    private String email;
    private String password;

    public User() {
        // TODO Auto-generated constructor stub
    }

    public User(String nom, String prenom, String email, String password) {
        this.nom = nom;
        this.prenom = prenom;
        this.email = email;
        this.password = password;
    }

    public String getNom() {
        return nom;
    }

    public void setNom(String nom) {
        this.nom = nom;
    }

    public String getPrenom() {
        return prenom;
    }

    public void setPrenom(String prenom) {
        this.prenom = prenom;
    }

    public String getEmail() {
        return email;
    }

    public void setEmail(String email) {
        this.email = email;
    }

    public String getPassword() {
        return password;
    }

    public void setPassword(String password) {
        this.password = password;
    }

    public String getFullName() {
        return this.prenom + " " + this.nom;
    }
}
```

FIGURE 1.1 – Code source de la classe `User.java`

## 1.3 Couche d'Accès aux Données (DAO)

### 1.3.1 Interface `IUser` et Implémentation `UserDaoImpl`

Au lieu de coupler fortement la technologie de stockage à notre logique, nous exploitons le pattern de conception "Data Access Object" (DAO). L'interface `IUser.java` établit un contrat standard (Trouver un utilisateur par email, vérifier son mot de passe, ou ajouter un utilisateur).

Son implémentation `UserDaoImpl.java` gère le bas niveau :

- Amorçage d'une connexion `JDBC`.
- Préparation des requêtes SQL paramétrées (`PreparedStatement`) pour contrer l'injection SQL.
- Exécution (`executeQuery` ou `executeUpdate`) et conversion du `ResultSet` retourné en un objet Java `User`.

```java
package dao;

import java.util.Map;
import java.util.concurrent.ConcurrentHashMap;

import models.User;

public class UserDaoImpl implements IUser {

    private static Map<String, User> userMap = new ConcurrentHashMap<>();

    @Override
    public void save(User user) {
        userMap.put(user.getEmail(), user);
    }

    @Override
    public User findByEmail(String email) {
        return userMap.get(email);
    }
}

```

FIGURE 1.2 – Exemple d'accès aux données via JDBC dans `UserDaoImpl`

# Chapitre 2

# Sécurité, Services et Sessions

## 2.1 Couche Métier (Services)

### 2.1.1 `UserService.java`

Indépendante des objets de session HTTP, cette classe effectue les traitements nécessaires avant la sollicitation de la base de données. Elle orchestre la validation des champs de connexion ou d'inscription, vérifie les règles métiers et passe le relais à la couche DAO appropriée.

```java
package metiers;

import dao.IUser;
import dao.UserDaoImpl;
import models.User;

public class UserService {

    private IUser userdao = new UserDaoImpl();

    public void register(User user) {
        userdao.save(user);
    }

    public User authenticate(String email, String password) {
        User user = userdao.findByEmail(email);
        if(user != null && user.getPassword().equals(password)) {
            return user;
        }
        return null;
    }
}

```

FIGURE 2.1 – Classe `UserService` : Logique métier

## 2.2 Composants Transversaux : Filtres et Listeners

La gestion sécurisée ne s'appuie pas juste sur les vues, mais sur des agents d'interception back-end en arrière-plan.

### 2.2.1 Filtre de Sécurité : `AuthFilter.java`

Implémentant l'interface `Filter`, cet outil intercepte les accès aux ressources protégées, typiquement les pages `.jsp` situées dans des zones privées (comme le tableau de bord). Le filtre exécute `HttpServletRequest.getSession(false)` pour vérifier si l'utilisateur possède un attribut de session actif. S'il n'est pas authentifié, il le redirige obligatoirement vers `login.jsp`.

```java
public void doFilter(ServletRequest request, ServletResponse response, FilterChain chain)
            throws IOException, ServletException {

        HttpServletRequest req = (HttpServletRequest) request;
        HttpServletResponse res = (HttpServletResponse) response;
        String uri = req.getRequestURI();

        boolean isIndex = uri.endsWith("/TP3-JEE/") || uri.endsWith("index.jsp");
        boolean isLoginPage = uri.endsWith("login.jsp");
        boolean isRegisterPage = uri.endsWith("register.jsp");
        boolean isLoginServlet = uri.endsWith("/login");
        boolean isRegisterServlet = uri.endsWith("/register");

        if (isLoginPage || isRegisterPage || isLoginServlet || isRegisterServlet || isIndex) {
            chain.doFilter(request, response);
            return;
        }

        HttpSession session = req.getSession(false);

        if (session != null && session.getAttribute("user") != null) {
            chain.doFilter(request, response);
        } else {
            res.sendRedirect("login.jsp");
        }
    }
```

FIGURE 2.2 – Interception des requêtes avec `AuthFilter`

### 2.2.2 Écouteur d'Application : `AppListener.java`

Étant un `ServletContextListener`, ce composant observe le démarrage de l'application sur le serveur web. Il se charge de configurer certaines propriétés globales ou d'initialiser d'éventuels pools de connexion base de données à l'instant où l'application est déployée.

## 2.3 Couche Contrôleur et Vues Web

Les Servlets lient les actions des formulaires HTML aux services Java.

### 2.3.1 Servlets d'Authentification

— `LoginServlet.java` : Reçoit le POST depuis `login.jsp` et vérifie les identifiants avec `UserService`. En cas de succès, elle crée la *Session* HTTP avec `request.getSession(true)` et transmet vers l'espace sécurisé.

— `LogoutServlet.java` : Met fin à la session instantanément avec la commande `session.invalidate()` pour verrouiller l'accès.

```java
protected void doPost(HttpServletRequest request, HttpServletResponse response) throws ServletException, IOException {
        request.setCharacterEncoding("UTF-8");
        String email = request.getParameter("email");
        String password = request.getParameter("password");

        if(email.isEmpty() || password.isEmpty()) {
            request.setAttribute("error", "Ooops, erreur !!! Vous devez remplir tous les champs");
            request.getRequestDispatcher("/login.jsp").forward(request,response);
        } else {
            UserService metier = (UserService) getServletContext().getAttribute("userService");
            User user = metier.authenticate(email, password);
            if (user != null) {
                HttpSession session = request.getSession();
                session.setAttribute("user", user);
                response.sendRedirect("dashboard.jsp");
            } else {
                request.setAttribute("error", "Email ou mot de passe incorrect");
                request.getRequestDispatcher("/login.jsp").forward(request, response);
            }
        }
    }
```

FIGURE 2.3 – Génération d'une session HTTP par la `LoginServlet`

### 2.3.2 Interfaces Visuelles (JSP)

Voici les interfaces conçues pour permettre l'interaction et la restriction d'accès aux utilisateurs.

#### Formulaires d'Inscription et de Connexion

Les pages `register.jsp` et `login.jsp` proposent des formulaires sécurisés permettant de s'inscrire puis de s'authentifier dans l'application web.

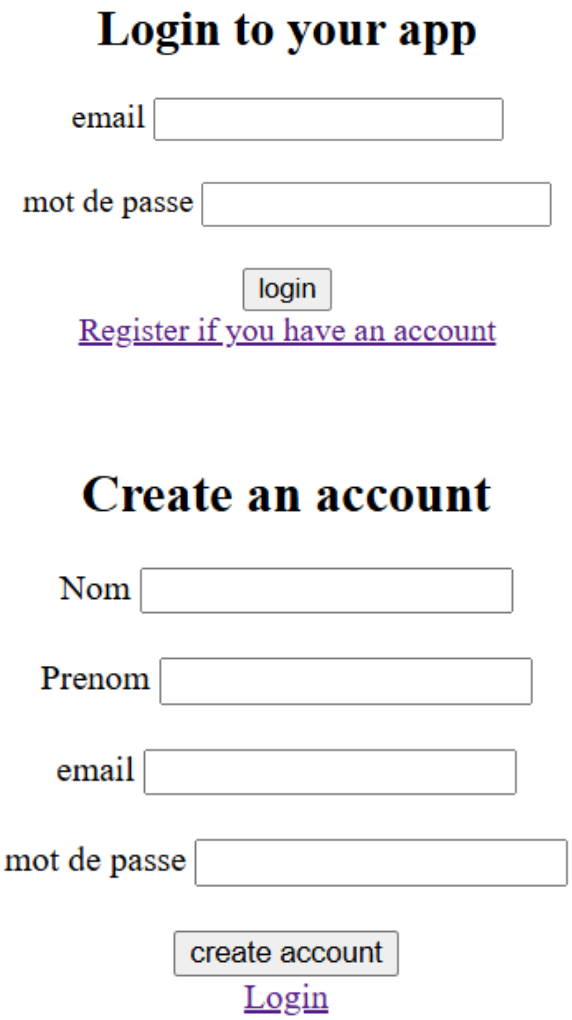


FIGURE 2.4 – Interfaces d'inscription et de vérification d'identité

**Espace Privé :** `dashboard.jsp`

Cet espace est inaccessible sans connexion. Son accès est vérifié en permanence par `AuthFilter`. Une fois dedans, il présente les fonctionnalités du tableau de bord tout en affichant un accueil dynamique tel que "Bonjour, ${user.nom}".

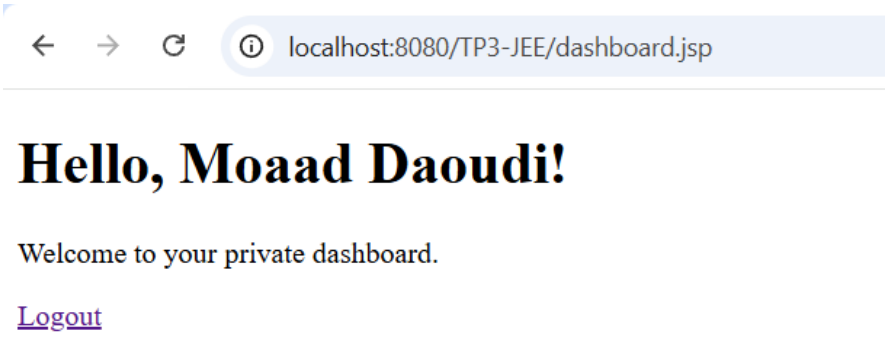


FIGURE 2.5 – Le tableau de bord sécurisé (Dashboard)

# Conclusion

L'implémentation de ce TP 3 garantit une maîtrise des piliers logiciels incontournables dans JEE : interaction via JDBC, sécurisation par implémentation d'un patron DAO et isolation des requêtes privées avec l'interface Filter. Ces briques solides assurent une conception orientée objet sécurisée et adaptable.